- there are now two 1 inputs to the bottom-right NAND gate
- so the Q' output becomes 0.

This shows how the J input is a set input. A similar argument shows that the K is a clear input. In this respect the JK flip-flop behaves in a similar way to the SR flip-flop as a storage device for one bit. However, there is an important difference in that if both J and K are input as a 1 then the values for Q and Q' are toggled (they switch value). This makes the JK flip-flop a more reliable device because there is no combination of input states that leave uncertainty as to which values are stored. The significant part of the truth table is shown as Table 19.04.

J	K	Clock	Q
0	0	↑	Q unchanged
1	0	↑	1
0	1	↑	0
1	1	↑	Q toggles

Table 19.04 Part of the truth table for a JK flip-flop

Question 19.02

Can you follow the logic in the JK flip-flop circuit to see that the truth table is an accurate representation of the logic for all combinations of input?

19.03 Boolean algebra basics

Chapter 4 (Section 4.01) introduced logic expressions consisting of logic propositions combined using Boolean operators. Boolean algebra provides a simplified way of writing a logic expression and a set of rules for manipulating an expression.

Whenever a form of algebra is used it is vital that there is an understanding of its meaning. As a simple example we can consider the following four interpretations of the meaning of 1 + 1:

```
1 + 1 = 2
1 + 1 = 10
1 + 1 = 0
1 + 1 = 1
```

The first shows denary arithmetic, the second binary arithmetic and the third bit arithmetic. The last one applies if Boolean algebra is being used. This is because in Boolean algebra 1 represents TRUE, 0 represents FALSE, and + represents OR. Therefore the fourth statement represents the logic statement:

TRUE OR TRUE is TRUE

There are options for the representation of Boolean algebra. For example, the symbols for AND and OR are sometimes represented as ∧ and ∨. There is also the option of writing A.B or AB for A AND B. The dot notation is used in this book. Finally, there are options for how NOT A (the inverse of A) can be represented. $\bar{A}$ is used here.

TIP

You do not need to remember the ∧ and ∨ notation.

Having established the notation for Boolean algebra we have to consider the rules that apply. These can formally be described as 'laws' or 'identities'. Table 19.05 contains a full listing.

Identity/Law	AND form	OR form
Identity	$1.A = A$	$0 + A = A$
Null	$0.A = 0$	$1 + A = 1$
Idempotent	$A.A = A$	$A + A = A$
Inverse	$A.\overline{A} = 0$	$A + \overline{A} = 1$
Commutative	$A.B = B.A$	$A + B = B + A$
Associative	$(A.B).C = A.(B.C)$	$(A + B) + C = A + (B + C)$
Distributive	$A + B.C = (A + B).(A + C)$	$A.(B + C) = A.B + A.C$
Absorption	$A.(A + B) = A$	$A + A.B = A$
De Morgan's	$\overline{A.B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A}.\overline{B}$
Double Complement	$\overline{\overline{A}} = A$	

Table 19.05 Boolean algebra identities (laws)

Some of the names used for the identities may be unfamiliar to you. This is not a concern. You should note that for all but one of the identities there is an AND form and an OR form. Furthermore, it is important to note that an identity written in one form can be transformed into the other by interchanging each 0 or 1 and each AND or OR. For example, $0.A = 0$ which reads FALSE AND A is FALSE transforms into TRUE OR A is TRUE, written in the algebra as $1 + A = 1$.

It can also be seen that some of the identities look like those applying in normal algebra with AND functioning as multiplication and OR functioning as addition. So you can use the terms 'product' and 'sum' in the context of Boolean algebra.

TASK 19.03

It is vital that you can interpret a Boolean expression correctly. Go through Table 19.05 item by item and in each case read out the full meaning. For example:

$1 + A = 1$ can be read as 'one plus A equals 1'

but must be understood as 'TRUE OR A is TRUE'.

Although De Morgan's laws look complicated at first glance, they can be rationalised easily. The inverse of a Boolean product becomes the sum of the inverses of the individual values in the product. The inverse of a Boolean sum is the product of the individual inverses.

Unfortunately, using the algebra to simplify expressions is not something which can be learnt as a routine. It requires lateral thinking as Worked Example 19.01 will show.

WORKED EXAMPLE 19.01

Using Boolean algebra to simplify an expression

Let's consider a simple example:

$A + \overline{A}.B$ can be simplified to $A + B$

In order to simplify the expression, we have to first make it more complicated! This is where the lateral thinking comes in. The OR form of the absorption identity is $A + A.B = A$. This can be used in reverse to replace A by $A + A.B$ to produce the following:

$$A + A.B + \overline{A}.B$$

We can for the moment ignore the A.

Applying the AND form of the commutative law and the OR form of the distributive law in reverse we can see that:

$A.B + \overline{A}.B$ is the same as $B.A + B.\overline{A}$ which converts to $B.(A + \overline{A})$

This allows us to use the OR form of the inverse identity which converts $A + \overline{A}$ into 1. As a result, the full expression, including the A that was ignored, has become:

$$A + B.1$$

The AND form of the identity law can now be applied to the B.1 term to change it to B so the expression then becomes $A + B$.

19.04 Boolean algebra applications

Creating a Boolean algebra expression directly from a truth table

One formal approach to creating a Boolean algebra expression for a particular problem is to start with the truth table and apply the sum of products method. This establishes a 'minterm' for each row of the table that results in a 1 for the output. This can be illustrated using the half adder circuit truth table shown in Figure 19.01. The only row of the table creating a 1 output for C has a 1 input for A and for B. The product becomes A.B and the sum of products has only this one term so we have:

$$C = A.B$$

For the S output, there are two rows that produce a 1 output so there is a sum of products containing two minterms:

$$S = \overline{A}.B + A.\overline{B}$$

Note that the 0 in a row is represented by the inverse of the input symbol.

In certain cases Boolean algebra laws can be applied directly to parts of the truth table. For example, consider a truth table which contains the following two rows:

A	B	C	X
0	1	0	1
0	1	1	1

The output for X is the same whether C = 0 or C = 1. There must therefore be a minterm which has no reference to C which in this case will be $\overline{A}.B$.

The Boolean algebra representation of a logic circuit

This approach can also be used as part of the process of creating a Boolean algebra logic expression from a circuit diagram. The truth tables for the individual logic gates are used and then some algebraic simplification is applied.

WORKED EXAMPLE 19.02

Creating a Boolean algebra logic expression for a half adder circuit

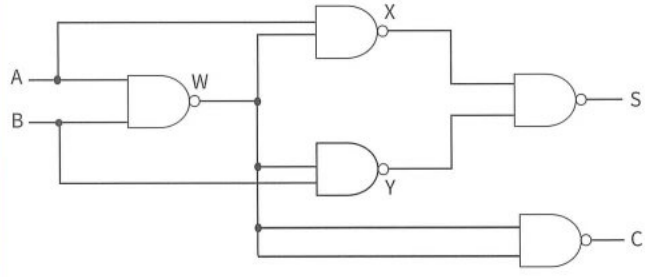


Figure 19.07 A half adder circuit

Figure 19.07 shows inputs A and B to a NAND gate with output W. The first three rows of the NAND truth table produce a 1 output so the sum of products has three minterms:

$$W = \overline{A}.\overline{B} + \overline{A}.B + A.\overline{B}$$

We can now consider the input of W to a NAND gate with A as the other input to produce the X output. Instead of using the NAND truth table we will use the fact that the NAND gate operates as an AND gate followed by a NOT gate.

The result of the AND operation is the product of the inputs so we get the following expression as output:

$$A.(\overline{A}.\overline{B} + \overline{A}.B + A.\overline{B})$$

This can be simplified by applying the distributive and inverse laws to give:

$$0 + 0 + A.A.\overline{B} \text{ which is simply } A.\overline{B}$$

To complete the NAND operation and get the value for X we have to apply the NOT operation which means we have to take the inverse of the above expression.

This is where we need the AND version of De Morgan's law which transforms the $A.\overline{B}$ into $\overline{A} + B$. So we have:

$$X = \overline{A} + B$$

The same laws applied to the output Y from the other intermediate NAND gate to give

$$Y = A + \overline{B}$$

Finally, we need to consider $A + \overline{B}$ and $\overline{A} + B$ being input to the final NAND gate. Again we can consider the AND operation first as the product of the inputs to produce the expression:

$$(A + \overline{B}).(\overline{A} + B)$$

We will not multiply this out; we will instead apply De Morgan's law directly to the expression to perform the inverse operation to complete the NAND operation. This gives:

$$S = \overline{A}.B + A.\overline{B}$$

This is the value obtained directly from the truth table so the algebra has been used correctly.